BLOQUE III - TEMA 6

DESARROLLO DE SISTEMAS

Arquitectura de sistemas cliente/servidor y multicapas: componentes y operación. Arquitecturas de servicios web y protocolos asociados

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. SISTEMAS DISTRIBUIDOS 5

1.1 Características clave de los sistemas distribuidos 6

2. MODELO CLIENTE/SERVIDOR 7

2.1 Concepto y características 7

2.2 Tipos de clientes 7

2.3 Comunicación y gestión de peticiones 8

3. ARQUITECTURA EN CAPAS Y MULTICAPAS 10

3.1. Arquitectura en 2 capas (cliente/servidor) 10

3.2. Arquitectura en 3 capas 11

3.3. Arquitectura en n capas y microservicios 11

4. COMPONENTES Y OPERACIÓN DEL SISTEMA 12

4.1. Servidor Web de aplicaciones 12

4.1.1. Servidor HTTP/SSL (TLS) 12

4.1.2. Servlets: Procesamiento del lado del servidor 13

4.1.3. Páginas dinámicas: Generación de contenido en tiempo real 13

4.1.4. JSP (Java Server Pages) y ASP (Active Server Pages) 13

4.1.5. Intérprete de scripting en el servidor 14

4.1.6. Frameworks de desarrollo de la lógica de negocio 14

4.2 Patrones comunes 14

4.2.1. DAO (Data Access Object) 15

4.2.2. Modelo Vista-Controlador (MVC) 15

4.4. Monitorización y gestión del rendimiento 16

5. ARQUITECTURAS DE SERVICIOS WEB 17

5.1. Servicios web 17

5.2. Servicios web en arquitecturas orientadas a servicios (SOA) 17

5.3. Servicios SOAP 19

5.4. REST 22

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5.5. Microservicios 23
5.6. Herramientas para documentar servicios web 23
6. RESUMEN PROTOCOLOS ASOCIADOS A SERVICIOS WEB 25
6.1 HTTP y HTTPS 25
6.2 XML, JSON y YAML 26
6.3 WSDL (Web Services Description Language) 26
6.4 UDDI (Universal Description, Discovery and Integration) 27
6.5. Repaso SOAP y REST 27
6.6 Protocolo gRPC y alternativas modernas 28
7. SEGURIDAD EN SISTEMAS DISTRIBUIDOS 28
7.1 Autenticación y autorización en servicios web 28
7.2 HTTPS, certificados digitales y SSL/TLS 29
7.3 API Keys, OAuth2 y JWT 29
8. TENDENCIAS ACTUALES EN ARQUITECTURA DE SISTEMAS ... 31
8.1 Arquitectura basada en microservicios 31
8.2 Serverless y funciones como servicio (FaaS) 31
8.3 Contenedores y orquestadores 32
8.3.1 Docker 32
8.3.2 Kubernetes 32
8.4 Service Fabric 33

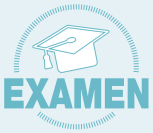
Significado de los iconos que aparecen dentro de los TEMAS:

EXAMEN Examen Importante Recordatorio Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. SISTEMAS DISTRIBUIDOS

En el desarrollo de sistemas modernos, especialmente en entornos empresariales, se ha pasado del modelo tradicional monolítico a **modelos distribuidos** que permiten mayor flexibilidad, escalabilidad y resiliencia. Las arquitecturas distribuidas permiten que una aplicación se ejecute **coordinadamente en múltiples nodos o componentes separados**, que pueden estar ubicados en distintos servidores o incluso en diferentes ubicaciones geográficas.



Un **sistema distribuido** es un conjunto de componentes software que se **ejecutan en distintos equipos o entornos físicos**, pero que cooperan para proporcionar una funcionalidad unificada al usuario final. Estos sistemas se diseñan de modo que los **módulos estén desacoplados y se comuniquen entre sí mediante redes**, servicios o protocolos estándar.

Objetivos principales:

- **Escalabilidad**: poder aumentar la capacidad del sistema añadiendo más nodos o recursos.
- **Disponibilidad**: permitir que el sistema siga funcionando incluso si fallan algunos componentes.
- **Mantenibilidad**: facilitar el despliegue, la actualización o sustitución de partes del sistema sin afectar al conjunto.
- **Desempeño**: optimizar el uso de recursos, permitiendo paralelismo y balanceo de carga.
- **Desacoplamiento funcional**: separar las responsabilidades en distintas capas o servicios (presentación, lógica, datos, etc.).

Las arquitecturas distribuidas se aplican ampliamente en organizaciones públicas y privadas para modernizar servicios, aumentar la disponibilidad y permitir la escalabilidad de sus aplicaciones.

- **Portales web de atención ciudadana**: distribuyen la lógica de autenticación, trámites, notificaciones y pagos en servicios independientes.
- **Sistemas de gestión empresarial (ERP)**: separan la gestión financiera, recursos humanos, compras y ventas en módulos intercomunicados.
- **Plataformas de comercio electrónico**: utilizan microservicios para productos, pedidos, usuarios, pagos y logística.
- **Servicios sanitarios**: integran historiales clínicos, citas, recetas y facturación en subsistemas distribuidos.
- **Aplicaciones móviles empresariales**: que se conectan a APIs REST distribuidas en la nube.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.1 CARACTERÍSTICAS CLAVE DE LOS SISTEMAS DISTRIBUIDOS

Los sistemas distribuidos presentan un conjunto de **características técnicas distintivas** que los diferencian de las arquitecturas monolíticas y que permiten su adaptación a entornos complejos, escalables y de alta disponibilidad.

a) Escalabilidad

La **escalabilidad** es la capacidad del sistema para **aumentar su rendimiento** de forma proporcional a la cantidad de recursos añadidos (por ejemplo, más nodos o servidores).

- **Escalabilidad horizontal (scale-out)**: se añaden más máquinas al sistema.
- **Escalabilidad vertical (scale-up)**: se aumentan los recursos de un nodo individual.

b) Transparencia

Un sistema distribuido debe intentar ofrecer distintas formas de **transparencia**, de modo que los usuarios y desarrolladores no tengan que preocuparse de la distribución física de los recursos.

Tipo de transparencia	Descripción
De acceso	Los recursos remotos se acceden como si fueran locales.
De ubicación	No es necesario conocer la localización física de un recurso.
De concurrencia	Varios procesos pueden acceder a un recurso compartido sin interferencias.
De replicación	El sistema puede replicar datos para disponibilidad sin que el usuario lo note.
De fallos	El sistema sigue funcionando a pesar de fallos en algunos componentes.
De movilidad	Los recursos o procesos pueden cambiar de ubicación sin afectar al sistema.

c) Concurrencia

Los sistemas distribuidos permiten que múltiples procesos o usuarios **interactúen simultáneamente** con los recursos del sistema.

- Se requiere gestión de sincronización y consistencia.
- Mejora la eficiencia y el aprovechamiento de recursos.

d) Tolerancia a fallos

Dado que los sistemas distribuidos suelen operar en **entornos complejos y heterogéneos**, deben estar preparados para **continuar funcionando incluso si algunos de sus componentes fallan**.

- Uso de **mecanismos de redundancia**, **replicación** y **reconfiguración automática**.

e) Independencia de heterogeneidad

Un sistema distribuido puede estar formado por **equipos con diferentes sistemas operativos, hardware, lenguajes o redes**, pero debe funcionar como una unidad coherente.

- Esto se consigue mediante el uso de **protocolos estándar** (HTTP, gRPC, SOAP) y **interfaces bien definidas** (APIs).

f) Compartición de recursos

Los sistemas distribuidos permiten que los recursos (archivos, bases de datos, servicios) **se compartan entre múltiples nodos** de forma controlada y eficiente.

- Requiere mecanismos de acceso concurrente, control de permisos y sincronización.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. MODELO CLIENTE/SERVIDOR

El modelo **cliente/servidor** es una de las arquitecturas más comunes en el desarrollo de sistemas distribuidos. Se basa en la separación de responsabilidades entre dos entidades: el **cliente**, que solicita un servicio o recurso, y el **servidor**, que lo proporciona.

2.1 CONCEPTO Y CARACTERÍSTICAS

El modelo cliente/servidor consiste en una arquitectura en la que una o varias aplicaciones cliente envían peticiones a una aplicación servidor, que las procesa y devuelve una respuesta.

Características principales:

- **Separación lógica de funciones**: el cliente consume el servicio, el servidor lo ejecuta.
- **Comunicación en red**: la interacción se realiza generalmente a través de protocolos como TCP/IP o HTTP.
- **Multicliente**: un servidor puede atender múltiples clientes simultáneamente.
- **Centralización del servicio**: la lógica principal del sistema reside en el servidor.
- **Escalabilidad horizontal**: se pueden agregar más servidores para distribuir la carga.

2.2 TIPOS DE CLIENTES

En función de la carga de procesamiento que asumen y su grado de dependencia del servidor, se distinguen tres tipos de clientes:

a) Cliente ligero (thin client)

- Se encarga exclusivamente de la **interfaz de usuario**.
- La lógica de negocio y el acceso a datos residen en el servidor.
- Ejemplo: navegador web accediendo a una aplicación web.

b) Cliente pesado (thick client)

- Incorpora parte o toda la lógica de negocio.
- Requiere menos comunicación con el servidor, pero es más complejo de actualizar.
- Ejemplo: aplicaciones de escritorio que acceden a bases de datos directamente.

c) Cliente híbrido

- Combina características de los anteriores: puede ejecutar lógica local y delegar procesos pesados al servidor.
- Muy común en aplicaciones móviles o modernas basadas en JavaScript con APIs REST.

d) Cliente rico / RIA (Rich Internet Application)

- Ejecuta una **interfaz de usuario compleja en el navegador web** utilizando tecnologías del lado del cliente (JS, AJAX, Angular, React).
- **Descarga trabajo del servidor** y mejora la interactividad, comunicándose **asíncronamente** con el backend.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Ejemplo: aplicaciones SPA (Single Page Application) desarrolladas con Angular o React.

Combinaciones comunes en arquitecturas cliente/servidor



Tipo de arquitectura	Descripción breve	N.º de capas típico
Cliente fino – Servidor grueso	El cliente solo muestra la interfaz; el servidor gestiona lógica y datos.	2 capas
Cliente pesado – Servidor ligero	El cliente contiene parte o toda la lógica de negocio y se conecta directamente a la base de datos.	2 capas
Cliente híbrido – Servidor intermedio	El cliente ejecuta parte de la lógica (ej. validaciones); el servidor intermedio actúa como API.	3 capas o más
Cliente web – Backend – Base de datos	Arquitectura clásica en 3 capas: presentación, lógica, persistencia.	3 capas
Cliente móvil – API Gateway – Microservicios	Arquitectura moderna distribuida, con clientes que se conectan a una capa de orquestación.	n capas (multicapa)
Cliente SPA (Single Page App) – API REST – Servicios	Lógica de presentación en el navegador, lógica y datos expuestos por APIs REST.	3 capas (desacopladas)

2.3 COMUNICACIÓN Y GESTIÓN DE PETICIONES

La interacción entre cliente y servidor se realiza a través de **protocolo de red**, y puede adoptar distintos patrones:

- **Síncrona**: el cliente espera la respuesta del servidor antes de continuar.
- **Asíncrona**: el cliente envía la petición y continúa con otras tareas hasta recibir la respuesta.
- **Protocolo HTTP/HTTPS**: ampliamente usado en aplicaciones web y móviles.
- **Protocolo TCP/IP**: en aplicaciones que requieren conexiones persistentes y eficientes.

Mecanismos comunes de comunicación:

- **Sockets**: nivel de transporte, conexión directa entre cliente y servidor.
- **Llamadas a procedimientos remotos (RPC)**: el cliente invoca funciones que se ejecutan en el servidor.
- **Servicios web (SOAP/REST)**: comunicación estructurada mediante XML o JSON.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Entorno	Cliente	Servidor
Aplicación web	Navegador (HTML + JS)	Servidor web + backend (ASP.NET, Java EE)
App móvil	App nativa (Android/iOS)	API RESTful en la nube
Aplicación de escritorio	Programa Windows/Linux/macOS	Servidor de base de datos o lógica remota
Gestión documental	Cliente web o app de escritorio	Servidor de contenidos (ECM, Alfresco)

Supera. by thePower
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. ARQUITECTURA EN CAPAS Y MULTICAPAS

Las **arquitecturas en capas** dividen una aplicación en bloques funcionales o niveles, de forma que cada capa se encarga de una responsabilidad concreta. Esta separación facilita el desarrollo, mantenimiento, escalabilidad y reutilización del software.

Una **arquitectura en capas** organiza el sistema en distintos niveles de responsabilidad bien definidos (presentación, negocio, datos, etc.). Cada capa **solo se comunica con la adyacente**, lo que reduce el acoplamiento y mejora la mantenibilidad.

- Separar las distintas funciones del sistema: interfaz de usuario, lógica de negocio, y almacenamiento de datos.
- Promover la reutilización de código.
- Facilitar la escalabilidad, mantenimiento y pruebas.

3.1. ARQUITECTURA EN 2 CAPAS (CLIENTE/SERVIDOR)

En la arquitectura **bicapas**, el sistema se divide en dos partes:

- **Capa cliente**: gestiona la interfaz de usuario y puede incluir parte de la lógica de negocio.
- **Capa servidor**: contiene la base de datos y el acceso a los datos.

Características:

- Comunicación directa entre cliente y base de datos.
- Rápida de implementar, pero difícil de escalar o mantener.
- Poco adecuada para entornos distribuidos complejos.



En una arquitectura cliente/servidor, es común referirse a los componentes con los términos frontend y backend.

- **Frontend** (cliente): es la parte de la aplicación con la que interactúa el usuario final. Se ejecuta en el dispositivo del cliente (navegador web, aplicación móvil, escritorio) y se encarga de la interfaz de usuario. *Ejemplo: una web desarrollada con HTML, CSS y JavaScript.*
- **Backend** (servidor): es la parte de la aplicación que se ejecuta en el servidor. Se encarga de la lógica de negocio, el acceso a datos y el procesamiento de las peticiones del cliente. *Ejemplo: una API REST desarrollada en Java, .NET o Python.*

Ambas partes se comunican normalmente a través de protocolos como HTTP/HTTPS.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.2. ARQUITECTURA EN 3 CAPAS

El **modelo de 3 capas** (también llamado **arquitectura en 3 capas**) es una forma de estructurar aplicaciones separando claramente las responsabilidades en tres niveles:

1. **Capa de presentación (UI)**: gestiona la interacción con el usuario (interfaz gráfica o web).
2. **Capa de lógica de negocio (BL)**: contiene las reglas, procesos y operaciones centrales de la aplicación.
3. **Capa de acceso a datos (DAL)**: gestiona la persistencia, recuperación y actualización de datos (bases de datos u otros almacenes).

Ventaja	Descripción
Permite separar responsabilidades en presentación, lógica y datos	Cada capa se especializa en una función concreta, lo que mejora el diseño.
Mejora la mantenibilidad y facilita el desarrollo colaborativo	Distintos equipos pueden trabajar en capas distintas con menor interferencia.
Escalabilidad más sencilla y eficiente	Las capas pueden desplegarse y escalarse de forma independiente.
Permite reutilizar la lógica de negocio desde distintas interfaces	Por ejemplo, una misma API puede servir a una app web, móvil y escritorio.

3.3. ARQUITECTURA EN N CAPAS Y MICROSERVICIOS

Cuando el sistema crece, pueden añadirse más capas o separar cada función en **módulos independientes**, dando lugar a arquitecturas **multicapas** o incluso **microservicios**.

En una **arquitectura de microservicios**, cada servicio es independiente, se despliega por separado y suele estar **identificado por un nombre único**, como una **URL o ruta de red**, que permite localizarlo y comunicarse con él mediante mecanismos de **descubrimiento de servicios** (por ejemplo, mediante un Service Registry como **Eureka** o **Consul**).

Arquitectura en n capas:

- Añade más niveles como servicios, integración, seguridad, cache, etc.
- Las capas pueden desplegarse en diferentes servidores.

Arquitectura de microservicios

- La lógica del sistema se divide en **módulos autónomos** que se comunican mediante APIs (REST, gRPC, mensajería).
- Cada microservicio puede tener su propia base de datos y estar desarrollado con tecnología distinta.
- Los microservicios pueden desarrollarse en **cualquier lenguaje de programación**, siempre que se comuniquen por medio de interfaces estándar (REST, gRPC, etc.).
- Favorece el despliegue independiente, la resiliencia y la escalabilidad granular.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4. COMPONENTES Y OPERACIÓN DEL SISTEMA

En las arquitecturas distribuidas y multicapas, el correcto funcionamiento del sistema depende de una serie de **componentes clave** que gestionan el procesamiento, la comunicación, la integración, la escalabilidad y el rendimiento general. Este apartado describe dichos elementos y su papel dentro del sistema.

4.1. SERVIDOR WEB DE APLICACIONES

Un **servidor de aplicaciones** es un software que proporciona un entorno de ejecución para aplicaciones, gestionando la lógica de negocio, la comunicación entre clientes y servidores, y la integración con bases de datos u otros sistemas externos. Su función principal es intermediar entre el cliente (navegador, aplicación móvil, etc.) y los recursos del servidor, facilitando la ejecución de aplicaciones dinámicas y distribuidas.

Se diferencian de un **servidor web**, que solo maneja peticiones HTTP y entrega contenido estático, ya que los servidores de aplicaciones procesan lógica compleja y gestionan sesiones, conexiones de bases de datos y ejecución de código en el servidor.

El desarrollo de aplicaciones web modernas se apoya en múltiples tecnologías que permiten la comunicación segura entre clientes y servidores, la ejecución de código dinámico y la implementación de la lógica de negocio en entornos distribuidos. Entre ellas destacan los servidores HTTP/SSL, los servlets, los lenguajes de scripting, las páginas dinámicas y los frameworks de desarrollo.

4.1.1. Servidor HTTP/SSL (TLS)

Un **servidor HTTP** es el software encargado de recibir y procesar las peticiones HTTP realizadas por los clientes (navegadores, aplicaciones móviles, APIs, etc.), **entregando respuestas en formato HTML, JSON u otros formatos.**

Para garantizar la **seguridad en la comunicación**, se utiliza el protocolo **SSL/TLS** (aunque se sigue llamando SSL, el estándar actual es **TLS – Transport Layer Security**). Este protocolo cifra la información intercambiada entre el cliente y el servidor, protegiéndola de ataques como la **intercepción de datos (Man-in-the-Middle, MitM)**.

Algunos de los servidores web más utilizados son:

- **Apache HTTP Server** → Servidor open-source ampliamente usado en entornos Linux.
- **Nginx** → Ofrece alta concurrencia y equilibrio de carga.
- **Microsoft IIS (Internet Information Services)** → Integrado con Windows Server.

El uso de **certificados SSL/TLS** en estos servidores es fundamental para la autenticación de sitios web y el cifrado de datos en aplicaciones críticas como banca en línea, comercio electrónico y administración pública.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.1.2. Servlets: Procesamiento del lado del servidor

Un **servlet** es una clase en el lenguaje Java utilizada para extender las capacidades de un servidor de aplicaciones. Su principal función es **procesar peticiones HTTP y generar respuestas dinámicas**, funcionando de manera similar a los **applets de Java**, pero ejecutándose en el servidor en lugar de en el navegador.

Funcionamiento de un Servlet en un servidor Java EE:

1. Un cliente envía una petición HTTP a un servidor de aplicaciones que ejecuta Java EE.
2. El servidor pasa la solicitud al servlet adecuado.
3. El servlet procesa la petición y puede interactuar con bases de datos u otros recursos.
4. Se genera una respuesta dinámica (HTML, JSON, XML) y se envía al cliente.

Los **servlets** se ejecutan en contenedores de servlets, como **Apache Tomcat**, **WildFly** o **GlassFish**.

4.1.3. Páginas dinámicas: Generación de contenido en tiempo real

Las **páginas dinámicas** permiten mezclar contenido estático (HTML, CSS) con código dinámico que se ejecuta en el servidor o en el cliente.

Se basan en una combinación de:

- **HTML estático** → Estructura fija de la página.
- **Código embebido** (JavaScript en el cliente o lenguajes como PHP en el servidor) → Permite interactividad y personalización del contenido.

Ejemplo de generación dinámica:

- Un usuario accede a una web de noticias y ve contenido generado en tiempo real basado en su ubicación o preferencias.
- Un e-commerce muestra productos recomendados según el historial de compras.

4.1.4. JSP (Java Server Pages) y ASP (Active Server Pages)

Los **lenguajes de scripting en el servidor** permiten generar páginas web de manera dinámica. Los más utilizados son:

- **JSP (Java Server Pages)**
  - Tecnología de Java EE que permite la inclusión de código Java dentro de páginas HTML.
  - Similar a los servlets, pero con una sintaxis más amigable y estructurada.
  - Se compila en un servlet antes de ser ejecutado por el servidor.

  Ejemplo de código JSP:

```
<%@ page language="java" %>
<html>
  <body>
    <h1>Bienvenido, <%= request.getParameter("usuario") %></h1>
  </body>
</html>
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **ASP (Active Server Pages)**
  - Lenguaje desarrollado por Microsoft para servidores IIS.
  - Permite la ejecución de scripts en VBScript o JavaScript.
  - A diferencia de JSP, **ASP es interpretado y no compilado**.

Ejemplo de código ASP:

```
<%
    Dim nombre
    nombre = Request.QueryString("usuario")
    Response.Write("Bienvenido " & nombre)
%>
```

Ambas tecnologías permiten generar páginas HTML dinámicas basadas en la entrada del usuario o en datos extraídos de bases de datos.

4.1.5. Intérprete de scripting en el servidor

Los **intérpretes de scripting** permiten la ejecución de scripts en el lado del servidor para generar contenido dinámico. Algunos ejemplos incluyen:

- **PHP** → Intérprete embebido en servidores Apache o Nginx para ejecutar código PHP.
- **Python** → Frameworks como Django y Flask usan Python como lenguaje de scripting en el servidor.
- **Node.js** → Ejecuta JavaScript en el servidor, permitiendo aplicaciones en tiempo real.

Los intérpretes facilitan la ejecución de código sin necesidad de compilación, permitiendo cambios rápidos en las aplicaciones web.

4.1.6. Frameworks de desarrollo de la lógica de negocio

Las aplicaciones web modernas siguen una arquitectura multicapa, separando la interfaz de usuario (front-end) de la lógica de negocio y la gestión de datos. Para ello, se emplean **frameworks de desarrollo**, que facilitan la estructuración del código y la gestión de dependencias.

Spring es un framework utilizado en aplicaciones web **Java EE**, ofreciendo herramientas para:

- **Gestión de dependencias** mediante inyección de componentes.
- **Desarrollo de APIs RESTful** con Spring Boot.
- **Conexión con bases de datos** mediante JPA/Hibernate.

Otros frameworks populares incluyen:

- **Django (Python)**
- **Express.js (Node.js)**
- **Laravel (PHP)**
- **ASP.NET Core (C#)**

4.2 PATRONES COMUNES

En las arquitecturas bien estructuradas se aplican **patrones de diseño** para separar responsabilidades y mejorar la mantenibilidad.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.2.1. DAO (Data Access Object)

El patrón **DAO** es un patrón de diseño estructural que tiene como objetivo **aislar la lógica de acceso a datos** del resto de la aplicación, especialmente de la **lógica de negocio**. Este patrón es muy utilizado en arquitecturas en capas (especialmente en la capa de acceso a datos) y en frameworks como Java EE, Spring o .NET.

Funciones del DAO:

- **Encapsular todas las operaciones de acceso a la base de datos** (consultas, inserciones, actualizaciones y eliminaciones).
- Proporcionar una **interfaz independiente de la tecnología de persistencia** (JDBC, JPA, Hibernate, ADO.NET...).
- Facilitar el **mantenimiento**, ya que los cambios en la forma de acceder a los datos (por ejemplo, cambio de motor de base de datos) se limitan a una clase DAO.
- Promover el **principio de responsabilidad única**: el DAO se encarga de acceder a los datos, y la lógica de negocio no necesita preocuparse por cómo se obtienen.

Ventajas del uso de DAO:

Ventaja	Descripción
Modularidad	Cambios en la base de datos no afectan a la lógica de negocio.
Reutilización	Los DAOs pueden ser utilizados por múltiples servicios.
Testabilidad	Fácil de sustituir por mocks o stubs en pruebas unitarias.
Mantenibilidad	El código está mejor organizado y segmentado.

4.2.2. Modelo Vista-Controlador (MVC)

El **modelo Vista-Controlador** es un **patrón de software de arquitectura** ampliamente adoptado en el desarrollo de aplicaciones web. Su propósito es **separar las responsabilidades** de una aplicación en tres componentes diferenciados, lo que facilita su **mantenimiento, escalabilidad y reutilización** .

Esta separación de capas es especialmente útil en aplicaciones web, ya que permite dividir el desarrollo entre diferentes perfiles (diseñadores, desarrolladores de lógica de negocio, administradores de bases de datos) y facilitar el trabajo en equipo.



Componentes del patrón MVC:

- **Modelo**: es el responsable de gestionar los datos y la lógica de negocio, incluyendo el acceso a la información (bases de datos, servicios, etc.).
  - Representa los datos y la lógica de negocio de la aplicación.
  - Gestiona el acceso a las bases de datos, validaciones, reglas y operaciones internas.
  - Es independiente de la presentación.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Vista**: Es la interfaz que se muestra al usuario.
  - Se encarga de representar visualmente los datos del modelo, generalmente utilizando HTML, CSS y plantillas.
  - No contiene lógica de negocio, solo presentación.
- **Controlador**: se encarga de recibir las entradas del usuario, interpretar las acciones y decidir qué hacer con ellas (por ejemplo, invocar métodos del modelo).
  - Actúa como intermediario entre la vista y el modelo.
  - Recibe las peticiones del usuario (por ejemplo, desde un formulario), las interpreta y determina qué acciones deben ejecutarse.
  - Invoca al modelo y selecciona la vista que debe mostrarse como respuesta.

Ejemplo de flujo MVC en una aplicación web:

1. El envía una petición (por ejemplo, rellenando y enviando un formulario al usuario).
5. El **controlador** recibe la petición, valida los datos y decide qué acción tomar.
6. El controlador llama al **modelo** para acceder o modificar la información.
7. El modelo responde con los datos procesados.
8. El controlador selecciona una **vista** apropiada y le pasa los datos necesarios.
9. La vista genera la respuesta visual que será enviada al navegador del usuario.

Aplicación del MVC en tecnologías web:

- En el lado del servidor: frameworks como **Spring MVC (Java)** , **ASP.NET MVC (C#)** , **Django (Python)** o **Laravel (PHP)** implementan este patrón de forma nativa.
- En el lado del cliente: frameworks JavaScript modernos como **Angular** , **React** (con su variante Flux o Redux), o **Vue.js** adoptan variantes del modelo MVC o arquitecturas similares para separar lógica, estado y presentación.

Variantes modernas del MVC

- **MVP (Model-View-Presenter):** el **Presentador** asume todo el control de la lógica de la interfaz de usuario y la Vista es **totalmente pasiva**.
- **MVVM (Model-View-ViewModel):** usa **Data Binding** (enlace de datos bidireccional) para comunicar la Vista con el **ViewModel** automáticamente. Muy usado en WPF y Angular.

4.4. MONITORIZACIÓN Y GESTIÓN DEL RENDIMIENTO

Es fundamental monitorizar los sistemas para detectar problemas, optimizar recursos y garantizar SLA (acuerdos de nivel de servicio).

Áreas a monitorizar:

- Uso de CPU, memoria, disco.
- Número de usuarios concurrentes.
- Tiempos de respuesta y latencia.
- Errores y excepciones.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Herramientas comunes:

- **Prometheus + Grafana**: métricas y dashboards.
- **Elastic + Kibana**: análisis de logs.
- **Application Insights (Azure)**, **New Relic**, **Dynatrace**: soluciones completas de APM (Application Performance Monitoring).

5. ARQUITECTURAS DE SERVICIOS WEB

Los **servicios web** son componentes que permiten la **interoperabilidad entre aplicaciones** distribuidas a través de una red, utilizando **protocolos estándar** y representaciones de datos comunes. Constituyen una parte esencial en arquitecturas distribuidas, orientadas a servicios y microservicios.

5.1. SERVICIOS WEB

Un **servicio web** es una funcionalidad accesible remotamente a través de una red (normalmente mediante HTTP/HTTPS), que permite el intercambio de información entre sistemas heterogéneos. Están diseñados para ser **plataforma-independientes**, **modulares** y **reutilizables**.

Características clave:

- Basados en e**stándares abiertos: XML, JSON, HTTP, SOAP, REST.**
- Comunicación entre aplicaciones independientemente del lenguaje o plataforma.
- Pueden ser públicos, privados o de uso interno.
- Usados en arquitecturas orientadas a servicios (SOA) y microservicios.

5.2. SERVICIOS WEB EN ARQUITECTURAS ORIENTADAS A SERVICIOS (SOA)

SOA (Service-Oriented Architecture) es un modelo arquitectónico que organiza el sistema en **servicios independientes y reutilizables**, que exponen una interfaz pública y se comunican mediante mensajes.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Características de SOA:

- Servicios autónomos que representan funcionalidades del negocio.
- Interacción mediante mensajes XML y protocolos estándar (frecuentemente SOAP).
- Integración con sistemas heredados.
- Fuerte acoplamiento con tecnologías como WSDL y UDDI.

Componentes principales de una arquitectura orientada a servicios (SOA)

La **arquitectura orientada a servicios (SOA)** es un estilo arquitectónico que organiza las funcionalidades de un sistema en **servicios autónomos, reutilizables e interoperables**, que se comunican entre sí a través de interfaces bien definidas, generalmente mediante mensajes estándar (XML, JSON, SOAP, etc.).

Uno de los casos más comunes de aplicación de SOA en la Administración Pública es la **Plataforma de Intermediación de Datos (PID)**, que permite compartir información entre organismos públicos a través de servicios expuestos y consumidos de forma estandarizada.

Componentes principales de SOA



Componente	Función
Servicio	Unidad funcional autocontenida que realiza una tarea específica.
Proveedor de servicios	Entidad que desarrolla, publica y mantiene el servicio.
Consumidor de servicios	Aplicación o sistema que utiliza (invoca) el servicio.
Registro de servicios	Repositorio que permite **publicar, descubrir y consultar** metadatos de servicios disponibles. Facilita el desacoplamiento entre proveedor y consumidor.

Ejemplo de flujo en SOA:

1. El **proveedor** registra un servicio en el **registro de servicios** (por ejemplo, mediante WSDL).
10. El **consumidor** consulta el registro, encuentra el servicio y lo consume mediante un mensaje estándar.
11. El **servicio** responde con la información solicitada o ejecuta una operación.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



- **SOA se implementa** con servicios en el **backend, usando tecnologías como SOAP, REST, WSDL, etc.**
- **En SOA los servicios son débilmente acoplados**, lo que mejora la flexibilidad y reutilización.
- La definición formal de los servicios en SOA se **realiza con WSDL (XML).**

5.3. SERVICIOS SOAP

SOAP (Simple Object Access Protocol) es un protocolo de mensajería estándar que permite el **intercambio estructurado de información entre aplicaciones** a través de una red, habitualmente mediante **HTTP o HTTPS**, aunque también puede funcionar sobre SMTP, JMS u otros protocolos de transporte.

El protocolo **SOAP** fue inicialmente una especificación mantenida por **Microsoft**, y más tarde fue adoptado como una **recomendación del W3C** en sus versiones tempranas. Sin embargo, la versión **SOAP 1.2** fue posteriormente publicada como un **estándar ISO/IEC**:

- **ISO/IEC 20022-1:2004** especifica SOAP como protocolo estándar para mensajería estructurada.

SOAP es ampliamente utilizado en entornos **empresariales y gubernamentales**, especialmente cuando se requieren capacidades avanzadas como **seguridad, transacciones, interoperabilidad entre sistemas heterogéneos** y comunicación fiable.

Estructura de un mensaje SOAP:

Un mensaje SOAP es un documento **XML bien formado** que contiene una serie de bloques bien definidos dentro de un elemento raíz **`<soap:Envelope>`.**

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <!-- Información opcional: seguridad, transacciones, control -->
  </soap:Header>
  <soap:Body>
    <!-- Mensaje de solicitud o respuesta -->
  </soap:Body>
</soap:Envelope>
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Componentes:

- **Envelope**: contenedor principal del mensaje.
- **Header** *(opcional)*: puede incluir información adicional como tokens de seguridad, metadatos o identificadores de transacción.
- **Body**: contiene los datos concretos de la solicitud o respuesta, normalmente estructurados como invocaciones a métodos y sus parámetros.

Componentes asociados a la arquitectura SOAP:

Componente	Función
WSDL (Web Services Description Language)	Lenguaje basado en XML que **describe la interfaz** del servicio web: métodos disponibles, tipos de datos, ubicación del servicio y protocolos utilizados.
UDDI (Universal Description, Discovery and Integration)	Sistema para **registrar y descubrir servicios web**. Permite buscar servicios disponibles por nombre, tipo o proveedor. Hoy en día está en desuso.
XML Schema (XSD)	Define la **estructura y tipos de datos** de los mensajes XML intercambiados, garantizando la validez de las peticiones y respuestas.

WS-Security (Web Services Security) es una especificación del estándar *WS- (Web Services)** que define cómo **firmar, cifrar y autenticar mensajes SOAP**. Es el mecanismo más utilizado para garantizar la **seguridad en servicios web SOAP**, y se usa ampliamente en la Administración Pública para garantizar integridad, autenticación y confidencialidad de los mensajes.

Funcionalidades de WS-Security:

- Firma digital de mensajes XML (basada en XML-Signature).
- Cifrado (XML-Encryption).
- Inclusión de tokens de seguridad (X.509, **SAML**).
  - **SAML (Security Assertion Markup Language)** se usa para intercambio de datos de autenticación y autorización, especialmente en entornos SSO (Single Sign-On).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Protección de cabeceras y cuerpo de mensajes SOAP.

A continuación, se recogen los principales estándares **WS-*** (Web Services Specifications) relacionados con la **seguridad en servicios web SOAP**, especialmente útiles en arquitecturas orientadas a servicios (SOA):



Estándar WS-*	Función principal	Resumen
WS-Security	Proporciona mecanismos de seguridad a nivel de mensaje SOAP	Define cómo firmar, cifrar y adjuntar tokens (usuario, SAML, etc.) a los mensajes SOAP.
WS-SecureConversation	Establece un **contexto de seguridad compartido** entre cliente y servidor	Permite reutilizar claves y reducir costes criptográficos en intercambios prolongados.
WS-Trust	Define cómo emitir, renovar y validar tokens de seguridad	Introduce el **Security Token Service (STS)**, que gestiona los tokens necesarios para WS.
WS-Federation	Extiende WS-Trust para federar identidades entre distintos dominios	Facilita el **SSO y la interoperabilidad** entre sistemas de identidad (federación de confianza).
WS-Policy	Permite describir **requisitos y capacidades** de seguridad de un servicio web	Por ejemplo, exigir firmas digitales o cierto tipo de token.
WS-Addressing	Establece cómo incluir **metadatos de direccionamiento** en mensajes SOAP	No trata de seguridad, pero es clave en mensajes con múltiples saltos.



WS-Security: seguridad básica (firma, cifrado, tokens).

WS-SecureConversation: "conversación segura" → contexto compartido.

WS-Trust: confianza → emisión y validación de tokens.

WS-Federation: identidad federada entre dominios.

WS-Policy: "reglas del juego" → políticas de seguridad.

Apache Axis2 es un **motor de servicios web** de código abierto desarrollado por la Apache Software Foundation. Es la **evolución de Apache Axis 1.x** y está diseñado para construir y consumir **servicios web SOAP** (y en menor medida, servicios RESTful).

Características principales de Axis2:

- Es una **plataforma para desarrollar y ejecutar servicios web**.
- Soporta los principales estándares de servicios web: **SOAP, WSDL, WS-Security, WS-ReliableMessaging**, etc.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Dispone de APIs tanto para **Java como para C**.
- Permite crear servicios web **a partir de clases Java (POJO)** o archivos WSDL.
- Ofrece un **modelo basado en módulos** para extender funcionalidades (por ejemplo, para seguridad o logging).
- Incluye su propio contenedor ligero de aplicaciones llamado **Axis2 Web Application (axis2.war)**.

5.4. REST

REST (Representational State Transfer) es un estilo de arquitectura que define un conjunto de restricciones para construir servicios web simples y escalables, utilizando los métodos estándar de HTTP.

Se trata de **estilo arquitectónico** propuesto por Roy Fielding en su tesis doctoral (2000), que define un conjunto de **restricciones** y principios para el diseño de sistemas distribuidos, especialmente servicios web escalables y modulares.

REST se ha convertido en el enfoque más extendido para la creación de **APIs web** modernas debido a su **simplicidad, eficiencia y compatibilidad con el protocolo HTTP**.

REST considera que todo en el sistema es un **recurso**, que se puede identificar de forma única mediante una URI (Uniform Resource Identifier), por ejemplo:

`https://api.ejemplo.com/usuarios/123`

Principios REST:

- **Identificación de recursos**: cada recurso tiene una URI única. Cada recurso (ej. usuario, producto, pedido) debe tener una **URI única**, como: `/api/clientes/123`
- **Métodos HTTP estándar**: Interacción mediante métodos estándar HTTP. REST utiliza los verbos HTTP para definir las operaciones sobre los recursos.
  - GET → obtener datos
  - POST → crear recursos
  - PUT → actualizar recursos
  - DELETE → eliminar recursos

Código	Significado
200	OK – Solicitud exitosa
201	Created – Recurso creado
204	No Content – Eliminación exitosa
400	Bad Request – Petición mal formada
401	Unauthorized – Autenticación fallida
404	Not Found – Recurso no encontrado
500	Internal Server Error – Error en servidor

- **Stateless**: Cada petición HTTP debe contener toda la información necesaria para que el servidor la procese. El servidor **no guarda el estado de la sesión**, lo que facilita la escalabilidad y distribución del sistema.
- **Formato ligero de representación**: intercambio de datos en JSON (preferido) o XML. También puede usarse **XML**, **YAML** u otros formatos, según necesidad.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Versionado de APIs**: Se recomienda incluir la versión en la URI o cabecera: /api/v1/clientes
- **Documentación clara con herramientas estándar**:
  - **Swagger/OpenAPI**: permiten generar documentación interactiva de forma automática.
  - Otras: Postman, Redoc, RAML.
- **HATEOAS (Hypermedia As The Engine Of Application State):** principio de REST por el cual las respuestas del servidor incluyen **hipervínculos** (hipermedios) que indican al cliente las **acciones y estados futuros disponibles**, permitiendo navegar la API dinámicamente sin conocimiento previo de las URL fijas. Es la restricción más avanzada de REST y corresponde al **nivel 3 del modelo de madurez de Richardson**.

Comparativa SOAP y REST

Característica	SOAP	REST
Estilo	Protocolo (basado en XML)	Arquitectura (basado en HTTP)
Formato de datos	XML	JSON (más habitual), XML
Transporte	No limitado (HTTP, SMTP, etc.)	HTTP
Estado	Puede ser stateful o stateless	Stateless
Seguridad avanzada	WS-Security	HTTPS + tokens (OAuth, JWT)
Facilidad de uso	Más complejo	Más sencillo y ligero
Casos de uso	Entornos empresariales, transacciones	APIs web, microservicios, móviles

5.5. MICROSERVICIOS

Los **microservicios** son una evolución moderna de los conceptos de SOA, con mayor énfasis en el **despliegue independiente**, la **ligereza** y la **automatización**.

Diferencias clave con SOA:

Aspecto	Servicios web tradicionales (SOA)	Microservicios
Granularidad	Menos granular, servicios más grandes	Servicios pequeños y especializados
Tecnología	SOAP/XML	REST/JSON, gRPC
Gestión y despliegue	Centralizado	Independiente por servicio
Escalabilidad	Escalado global	Escalado por componente
Comunicación	Normalmente síncrona	Síncrona o asíncrona
Dependencias	Acopladas	Desacopladas, orientadas a APIs

5.6. HERRAMIENTAS PARA DOCUMENTAR SERVICIOS WEB

1. Servicios web REST

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Los servicios web REST no tienen un estándar único de descripción como WSDL en SOAP, pero existen varias herramientas ampliamente utilizadas para **documentar y describir APIs REST**:

Herramienta / Lenguaje	Descripción breve
OpenAPI / Swagger	Especificación estándar para describir APIs REST. Permite generar documentación interactiva, clientes y servidores automáticamente.
RAML (RESTful API Modeling Language)	Lenguaje basado en YAML que permite describir de forma clara y estructurada una API REST.
API Blueprint	Lenguaje de marcado para describir APIs que se enfoca en la legibilidad humana.
Postman	Herramienta que, además de pruebas de APIs, permite generar documentación automática basada en las peticiones definidas.
Redoc	Visualizador de documentación OpenAPI de aspecto profesional, fácil de integrar en webs.
WHEN (condición) (opcional)	Permite ejecutar el trigger **solo si se cumple** una condición lógica.
BEGIN … END	Delimita el **bloque de instrucciones SQL** a ejecutar automáticamente.

2. Servicios web SOAP

Los servicios SOAP son más formales y estandarizados, y se documentan principalmente con:

Herramienta / Lenguaje	Descripción breve
WSDL (Web Services Description Language)	Lenguaje XML estándar para describir los servicios web SOAP: operaciones disponibles, tipos de datos, protocolos, etc.
XSD (XML Schema Definition)	Define los tipos de datos usados por los servicios SOAP que se describen en WSDL.

WADL (Web Application Description Language): es una especificación XML propuesta por Sun Microsystems para describir servicios REST (equivalente al WSDL de SOAP). No es un estándar oficial; el estándar de facto actual es Swagger/OpenAPI.

Elemento	Qué es	Uso
SOAP	Protocolo de mensajería basado en XML.	Intercambiar mensajes entre servicios web.
WSDL	Lenguaje XML de descripción de servicios.	Describir servicios SOAP: operaciones, mensajes y ubicación.
WADL	Lenguaje XML de descripción de servicios.	Describir servicios REST: recursos, métodos HTTP y representaciones.

WSIL (Web Services Inspection Language) es una especificación desarrollada por IBM y Microsoft que permite:

- **Publicar y descubrir** servicios web disponibles en un servidor o dominio.
- Proporcionar un documento XML que **describe las ubicaciones de los archivos WSDL** u otros descriptores de servicio web.
- Ofrecer un mecanismo **complementario o alternativo a UDDI** para el descubrimiento de servicios.

WSIL está pensado para ser más **ligero y directo** que los registros centralizados como UDDI, facilitando que los clientes inspeccionen dinámicamente los servicios que un servidor pone a disposición.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6. RESUMEN PROTOCOLOS ASOCIADOS A SERVICIOS WEB

Los servicios web utilizan un conjunto de **protocolos y formatos de datos estándar** que permiten la comunicación entre sistemas heterogéneos. Estos protocolos definen cómo se estructuran, transportan y describen los mensajes intercambiados entre cliente y servidor.

6.1 HTTP Y HTTPS

El **protocolo HTTP (HyperText Transfer Protocol)** es el principal protocolo de transporte utilizado por los servicios web. Se basa en el modelo cliente/servidor, donde el cliente realiza peticiones y el servidor responde con recursos.

Características de HTTP:

- Protocolos sin estado (stateless).
- Métodos estándar: GET, POST, PUT, DELETE, PATCH.
- Utiliza puertos: 80 para HTTP y 443 para HTTPS.

Idempotencia: un método es **idempotente** si hacer la misma petición varias veces tiene el mismo efecto en el servidor que hacerla una sola vez. **GET, PUT y DELETE son idempotentes**. **POST no es idempotente** (crea un recurso nuevo cada vez).

Método	¿Idempotente?	Efecto de repetir la petición
GET	Sí	Devuelve el mismo recurso, sin modificarlo.
PUT	Sí	El recurso queda en el mismo estado final.
DELETE	Sí	El recurso sigue eliminado.
POST	No	Crea un recurso nuevo cada vez.

HTTPS:

Es la versión **segura** de HTTP, que añade una capa de cifrado mediante **TLS (Transport Layer Security)** para proteger la confidencialidad e integridad de los datos transmitidos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



CORS (Cross-Origin Resource Sharing) es un mecanismo que permite o restringe las peticiones HTTP de origen cruzado hechas desde scripts en el navegador (por ejemplo, usando fetch() o XMLHttpRequest).

Cuando el frontend (cliente) está en un dominio distinto al backend (servidor), el navegador bloquea por defecto las peticiones por motivos de seguridad. Para permitirlas de forma controlada, el backend debe configurar cabeceras CORS que indiquen qué orígenes están permitidos.

6.2 XML, JSON Y YAML

Los servicios web necesitan representar datos en formatos estructurados que sean legibles por máquina y por humanos.

XML (Extensible Markup Language)

- Usado tradicionalmente en servicios SOAP.
- Auto-descriptivo, basado en etiquetas jerárquicas.
- Verboso y rígido, pero muy estructurado.

JSON (JavaScript Object Notation)

- Más ligero que XML, ideal para servicios REST.
- Formato basado en pares clave-valor.
- Rápido de procesar, ampliamente usado en aplicaciones web y móviles.

YAML (YAML Ain't Markup Language)

- Similar a JSON pero más legible para humanos.
- Utilizado en configuración (por ejemplo, Docker, Kubernetes).
- Poco usado en servicios web en tiempo real.

6.3 WSDL (WEB SERVICES DESCRIPTION LANGUAGE)

WSDL es un lenguaje basado en XML que describe la **interfaz pública** de un servicio web SOAP.

Define:

- Los **métodos disponibles** (operaciones).
- El formato y tipo de los mensajes.
- Los **protocolos y ubicaciones** del servicio.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipos de operaciones en WSDL

Tipo de operación	Descripción
one-way	El cliente envía un mensaje. El servicio **no responde**.
request-response	El cliente envía un mensaje y espera **una respuesta del servicio**.
solicit-response	El servicio envía un mensaje y espera **una respuesta del cliente**.
notification	El servicio envía un mensaje al cliente **sin esperar respuesta**.

6.4 UDDI (UNIVERSAL DESCRIPTION, DISCOVERY AND INTEGRATION)



UDDI fue un estándar propuesto para permitir el **registro y descubrimiento de servicios web** en entornos SOA.

Objetivo:

- Centralizar la publicación y búsqueda de servicios disponibles.
- Actuar como un "directorio telefónico" para servicios web.

Estado actual:

- Prácticamente en **desuso**.
- Reemplazado en la práctica por documentación en portales, catálogos API, o Swagger/OpenAPI para servicios REST.

6.5. REPASO SOAP Y REST

SOAP (Simple Object Access Protocol)

- Basado en XML.
- Rígido, bien estructurado.
- Soporta seguridad, transacciones, mensajería asíncrona.
- Requiere WSDL para describir servicios.

REST (Representational State Transfer)

- Basado en HTTP estándar.
- Ligero, flexible, usa JSON o XML.
- Utiliza verbos HTTP como método de operación.
- Se describe normalmente con OpenAPI (Swagger).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6.6 PROTOCOLO GRPC Y ALTERNATIVAS MODERNAS

gRPC (Google Remote Procedure Call) es un sistema moderno de comunicación remota diseñado para alto rendimiento y eficiencia.

Características de gRPC:

- Usa **Protocol Buffers (protobuf)** en lugar de XML o JSON.
- Permite comunicación **síncrona y asíncrona**.
- Soporta streaming bidireccional.
- Más rápido que REST para muchas operaciones.

Alternativas modernas:

Tecnología	Características principales
GraphQL	Permite a los clientes definir exactamente qué datos quieren.
WebSockets	Comunicación bidireccional y persistente (ideal para tiempo real).
Server-Sent Events (SSE)	Envío de datos desde el servidor al cliente de forma unidireccional y continua.

7. SEGURIDAD EN SISTEMAS DISTRIBUIDOS

En sistemas distribuidos, la seguridad es crítica, ya que los datos y servicios se transmiten y consumen a través de redes públicas o compartidas. La seguridad debe garantizar la **confidencialidad, integridad, autenticación, autorización** y **disponibilidad** de los servicios y datos.

7.1 AUTENTICACIÓN Y AUTORIZACIÓN EN SERVICIOS WEB

Autenticación: Es el proceso de verificar la identidad de un usuario o sistema que solicita acceso a un recurso.

Métodos comunes:

- Usuario/contraseña.
- Certificados digitales.
- Tokens de acceso.

Autorización: Es el proceso posterior a la autenticación, en el que se determina si el usuario tiene permiso para realizar una acción o acceder a un recurso.

Sistemas típicos de control de acceso:

- Basado en roles (RBAC).
- Basado en atributos (ABAC).
- Listas de control de acceso (ACLs).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7.2 HTTPS, CERTIFICADOS DIGITALES Y SSL/TLS

HTTPS (HTTP Secure)

Es la versión segura de HTTP. Añade una capa de cifrado para proteger los datos transmitidos entre cliente y servidor.

SSL/TLS

- **TLS (Transport Layer Security)** es el protocolo actual (SSL está obsoleto).
- Proporciona:
  - **Cifrado** de la información.
  - **Autenticación del servidor** (y opcionalmente del cliente).
  - **Integridad** del mensaje.

Certificados digitales

Emitidos por **Autoridades de Certificación (CA)**, permiten:

- Verificar la identidad del servidor.
- Establecer una conexión cifrada basada en confianza.

7.3 API KEYS, OAUTH2 Y JWT

Los sistemas distribuidos modernos, especialmente en entornos RESTful o microservicios, utilizan esquemas de autenticación y autorización modernos y flexibles.

API Keys

- Claves estáticas que permiten identificar y autorizar al cliente.
- Fáciles de implementar, pero menos seguras si no se usan sobre HTTPS.

OAuth2

- Protocolo estándar para **delegación de autorización**.
- Permite que un sistema obtenga acceso limitado a recursos de otro sin compartir credenciales.
- Usa **Access Tokens** que pueden tener permisos y duración limitados.
- **Principales flujos (grants) de OAuth2:**
  - **Authorization Code:** para aplicaciones de servidor (backend). Combinado con **PKCE** (Proof Key for Code Exchange), es también el flujo recomendado actualmente para SPA y aplicaciones móviles.
  - **Implicit:** (obsoleto/peligroso) para SPA. Ha sido sustituido por Authorization Code con PKCE.
  - **Client Credentials:** para comunicación de máquina a máquina (M2M) o microservicios.
  - **Resource Owner Password:** (obsoleto) intercambio directo de credenciales por token.

JWT (JSON Web Token)

- Formato de token compacto y seguro basado en JSON.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Compuesto por tres partes codificadas en base64: **header**, **payload** y **firma**.
- Se usa frecuentemente junto con OAuth2.
- Autocontenidos: el token puede incluir la información del usuario y permisos, evitando consultas repetidas al servidor.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8. TENDENCIAS ACTUALES EN ARQUITECTURA DE SISTEMAS

Las arquitecturas de sistemas están evolucionando rápidamente hacia modelos más modulares, escalables y orientados a la nube. Las siguientes tendencias reflejan la transición desde enfoques monolíticos hacia arquitecturas **distribuidas, desacopladas y gestionadas como servicios**.

8.1 ARQUITECTURA BASADA EN MICROSERVICIOS

La **arquitectura de microservicios** consiste en dividir una aplicación en múltiples **servicios pequeños e independientes**, que se comunican entre sí mediante **APIs ligeras** (normalmente REST o gRPC).

Características clave:

- Cada microservicio tiene su **propia lógica de negocio** y puede ser desplegado de forma independiente.
- Escalabilidad individual de cada componente.
- Aislamiento de fallos y mejora de la resiliencia.
- Permite el uso de diferentes tecnologías en cada servicio (poliglotismo).

Ventajas:

- Facilita el despliegue continuo y la automatización (DevOps).
- Reduce el acoplamiento entre equipos de desarrollo.
- Mejora la mantenibilidad a largo plazo.

8.2 SERVERLESS Y FUNCIONES COMO SERVICIO (FAAS)

Serverless es un modelo de computación en la nube donde el desarrollador sube funciones de negocio que se ejecutan **sin gestionar servidores**, escalando automáticamente según la demanda.

FaaS (Functions as a Service)

- El desarrollador implementa **funciones individuales** que responden a eventos (HTTP, colas, temporizadores...).
- El proveedor de nube (Azure Functions, AWS Lambda, Google Cloud Functions) gestiona la infraestructura.

Ventajas:

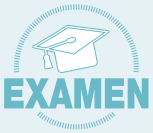
- Reducción de costes: se paga solo por el tiempo de ejecución real.
- Alta escalabilidad automática.
- Sin mantenimiento de infraestructura.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8.3 CONTENEDORES Y ORQUESTADORES

Los **contenedores** son entornos ligeros, portables y consistentes que agrupan una aplicación con todas sus **dependencias, bibliotecas y configuración**, garantizando su ejecución de forma uniforme en diferentes entornos (desarrollo, pruebas, producción). A diferencia de las máquinas virtuales, los contenedores **comparten el kernel del sistema operativo host**, lo que los hace más eficientes y rápidos de arrancar.

8.3.1 Docker



Docker es una plataforma de código abierto que permite **crear, empaquetar y desplegar aplicaciones en contenedores** de forma automatizada.

Un **contenedor Docker** es una instancia de una imagen, que es un paquete inmutable que incluye:

- El código de la apli cación.
- Dependencias y librerías necesarias.
- Configuraciones específicas.
- El entorno de ejecución (por ejemplo, un intérprete de Python o una máquina virtual de Java).

¿Por qué usar Docker?

- **Automatiza el empaquetado, despliegue y ejecución** de aplicaciones.
- **Asegura la portabilidad** entre entornos: lo que funciona en tu máquina funcionará en producción.
- **Facilita la integración con CI/CD** (Integración y Despliegue Continuo).
- **Aísla procesos**, aumentando la seguridad y evitando conflictos entre aplicaciones.
- **Compatible con orquestadores** como Kubernetes, Docker Swarm o OpenShift.

Herramientas en Docker:

- **Dockerfile**: archivo de texto con instrucciones para construir una imagen Docker.
- **docker build:** genera una imagen a partir de un Dockerfile.
- **docker run:** ejecuta un contenedor desde una imagen.
- **docker-compose:** herramienta para definir y ejecutar **múltiples contenedores** usando un archivo YAML.

8.3.2 Kubernetes

Kubernetes (K8s) es un sistema de **orquestación de contenedores** desarrollado originalmente por Google y ahora mantenido por la Cloud Native Computing Foundation (CNCF).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Permite gestionar **aplicaciones en contenedores** a gran escala de forma automatizada, incluyendo su despliegue, escalado, monitorización y recuperación ante errores.

Funcionalidades clave:

- **Balanceo de carga**: distribuye el tráfico entre los distintos contenedores disponibles.
- **Escalado horizontal automático**: adapta el número de réplicas de una aplicación según la carga.
- **Autorreparación (auto-healing)**: reinicia contenedores fallidos o los sustituye por otros sanos.
- **Gestión de configuración y secretos**: mediante ConfigMaps y Secrets.
- **Despliegue gradual**: permite realizar *rolling updates* y *rollbacks*.

Principales objetos de Kubernetes:

Objeto	Descripción
Pod	Unidad mínima de ejecución. Puede contener uno o más contenedores que comparten red y almacenamiento.
Service	Exposición de uno o varios Pods como un único punto de acceso de red.
Deployment	Controla el ciclo de vida de los Pods (creación, escalado, actualización).
Volume	Mecanismo para **almacenamiento persistente**, accesible desde los Pods.
Namespace	Permite **aislar recursos** dentro de un clúster (útil para separar entornos o equipos).

Herramientas del ecosistema:

- **Helm**: gestor de paquetes para Kubernetes (plantillas YAML reutilizables llamadas "charts").
- **Minikube**: entorno local para probar Kubernetes en un solo nodo.
- **kubectl**: herramienta de línea de comandos para interactuar con un clúster Kubernetes.

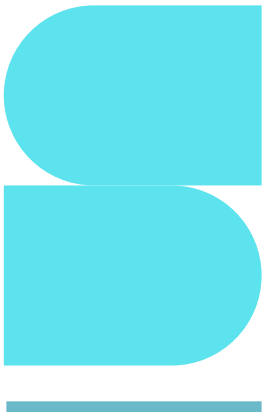
8.4 SERVICE FABRIC

Service Fabric es una **plataforma de orquestación de microservicios** desarrollada por **Microsoft**, diseñada para construir, desplegar y administrar **aplicaciones distribuidas escalables y resilientes**.

Es un **entorno de ejecución y gestión** que permite:

- Desplegar aplicaciones basadas en **microservicios o contenedores**.
- Ejecutarlas de forma **distribuida** en un clúster de nodos (máquinas físicas o virtuales).
- Supervisar su **estado, disponibilidad y escalado automático**.
- Gestionar **ciclo de vida**, **versionado**, **recuperación ante fallos** y **actualizaciones** sin interrupción.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.